SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU- 574 146

MICROSERVICES ARCHITECTURE & IMPLEMENTATION

MINIPROJECT REPORT

ON

" The Emergency Resource Locator and Response System"

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By,

DISHA J KOTIAN **01SU23CS059**
CHAITHALI R SHETTIGAR **01SU23CS048**

UNDER THE GUIDANCE OF

Mr. YOGESH

IBM TRAINER

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ABSTRACT

The Emergency Resource Locator and Response System is a web-based application designed to enhance emergency reporting and response coordination. The system aims to provide a centralized platform where users can report emergencies quickly and track their status in real time, while administrators can monitor, manage, and update emergency requests efficiently.
The application is developed using a microservices architecture, with React and TypeScript for the frontend and Node.js with Express and MongoDB for the backend. An API Gateway is used to manage communication between services, ensuring modularity and scalability. The system incorporates secure authentication using JSON Web Tokens (JWT) and role-based access control to ensure data security and controlled access.

Users can submit emergency details such as type, location, and description, which are stored and processed by the system. Administrators can view all reported emergencies, update their status, and analyze system data through a centralized dashboard. This improves communication, reduces response time, and ensures transparency in handling emergency situations.
The proposed system demonstrates the practical implementation of modern web technologies and distributed system design principles to address real-world emergency management challenges. Future enhancements may include real-time notifications, GPS-based tracking, and AI-based prioritization of emergencies.

CONTENTS

Chapter 1	**INTRODUCTION**	**1**
1.1	Overview of the Project	1
1.2	Problem Definition	1
1.3	Objectives of the Project	1
1.4	Scope and Applications	2
Chapter 2	**LITERATURE REVIEW**	**3**
2.1	Existing Systems	3
2.2	Proposed System	3
Chapter 3	**SYSTEM ANALYSIS AND DESIGN**	**4**
3.1	System Architecture	4
3.2	Hardware Requirements	5
3.3	Software Requirements	5
3.4	Use Case	5
Chapter 4	**METHODOLOGY**	**6**
4.1	Working Principle	6
4.2	Process Flow	6
4.3	Module Description	7
4.4	Error Handling and Validation	8
Chapter 5	**EXECUTION**	**9**
5.1	Execution Steps	9
5.2	Compilation & Execution Commands	9

Chapter 6	**RESULTS**	**10**
Chapter 7	**CONCLUSION AND FUTURE SCOPE**	**12**
7.1	Conclusion	12
7.2	Future Enhancements	12
Chapter 8	**REFERENCES**	**13**

LIST OF FIGURES

Figure Number	Title	Page Number
3.1.1	System Architecture	3
3.4.1	Use Case Diagram	5
4.2.1	Data Flow Diagram (Level 1)	7
4.4.1	Class Diagram	8
6.1.1	User login interface.	10
6.1.2	User registration interface	10
6.1.3	Emergency requests dashboard.	11
6.1.4	Emergency report submission form.	11

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

The Emergency Resource Locator and Response System is a full-stack web application designed to improve emergency reporting, monitoring, and response coordination. The system is developed using a microservices architecture with React and TypeScript for the frontend and Node.js with Express and MongoDB for the backend.

The platform enables users to report emergencies by specifying incident type, location, and description. Administrators can monitor incidents, update emergency statuses, and analyze system data through a centralized dashboard. The system ensures structured communication and real-time tracking of emergency situations.

The backend is divided into independent services:

- Auth Service handles registration, login, and token generation.
- Emergency Service manages the lifecycle of emergency requests.
- Admin Service provides administrative data and analytics.
- API Gateway routes requests between frontend and backend services

1.2 Problem Definition

Emergency reporting in many organizations is fragmented and inefficient due to reliance on manual methods such as phone calls, emails, and registers. These methods lack transparency, tracking, and accountability.

This results in delayed responses, poor communication, and lack of visibility of emergency status. The proposed system solves these problems by providing a centralized, digital, and role-based emergency management platform.

1.3 Objectives of the Project

The primary objectives of the system are:

- To provide secure authentication using JWT
- To enable quick emergency reporting
- To maintain lifecycle tracking of emergencies
- To provide administrative monitoring and control
- To design a scalable microservices-based system

1.4 Scope and Applications

Scope:

- Emergency reporting and tracking
- Role-based access control (User/Admin)
- Admin dashboard with analytics

Applications:

- Hospitals and healthcare systems
- Disaster management systems
- Emergency response services
- Residential and corporate environments

CHAPTER 2

LITERATURE REVIEW

2.1 Existing System

Existing systems include:

1. **Manual Systems**
   - Based on phone calls or registers
   - No tracking or record management
2. **Basic Web Portals**
   - Limited functionality
   - No real-time updates
3. **Monolithic Systems**
   - Hard to scale
   - Difficult to maintain

Limitations:

- No centralized tracking
- Poor scalability
- Weak security

2.2 Proposed System

The proposed system uses:

- Microservices architecture
- API Gateway
- JWT authentication
- Role-based access control

This improves scalability, security, and maintainability.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The system follows a layered architecture:

1. User interacts with frontend
2. Requests go to API Gateway
3. Gateway routes to respective services
4. Services process data and interact with MongoDB
5. Response is returned to frontend

Components:

- Auth Service
- Emergency Service
- Admin Service
- API Gateway

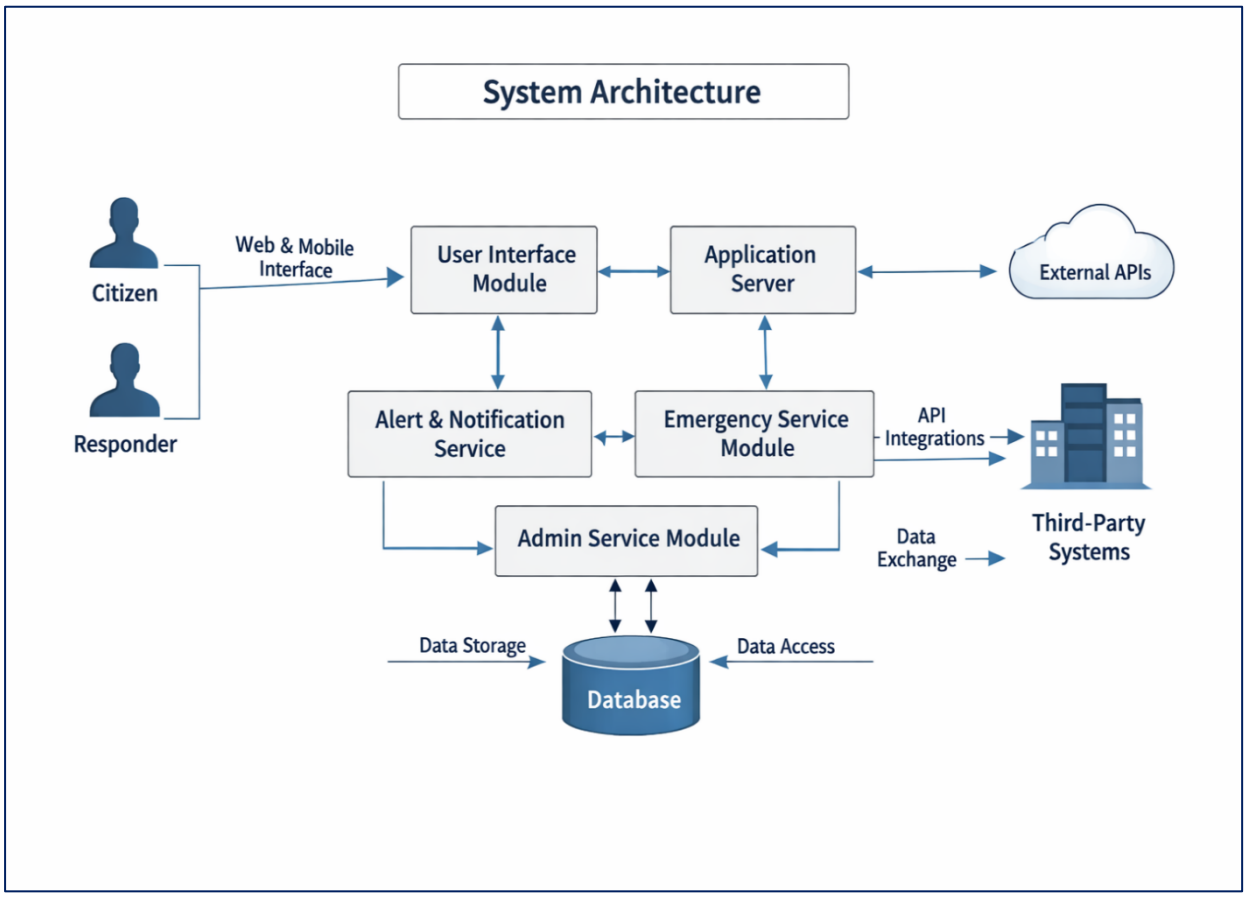

Figure 3.1.1 : System Architecture

3.2 Hardware Requirements

Processor	: Dual-core or higher
RAM	: Minimum 8 GB
Storage	: 10 GB or more
Input Devices	: Keyboard and Mouse
Display	: Monitor / Laptop Screen
Internet Connectivity	: Required

3.3 Software Requirements

- **Node.js** : Backend execution environment
- **MongoDB** : Flexible data storage
- **React + TypeScript** : Scalable frontend development
- **Express.js** : REST API framework
- **Tailwind CSS** : Efficient UI styling
- **JWT & bcrypt** : Secure user authentication
- **Vite** : Rapid development tooling

3.4 Use Case

The use case diagram illustrates the interactions between users and administrators with the system.

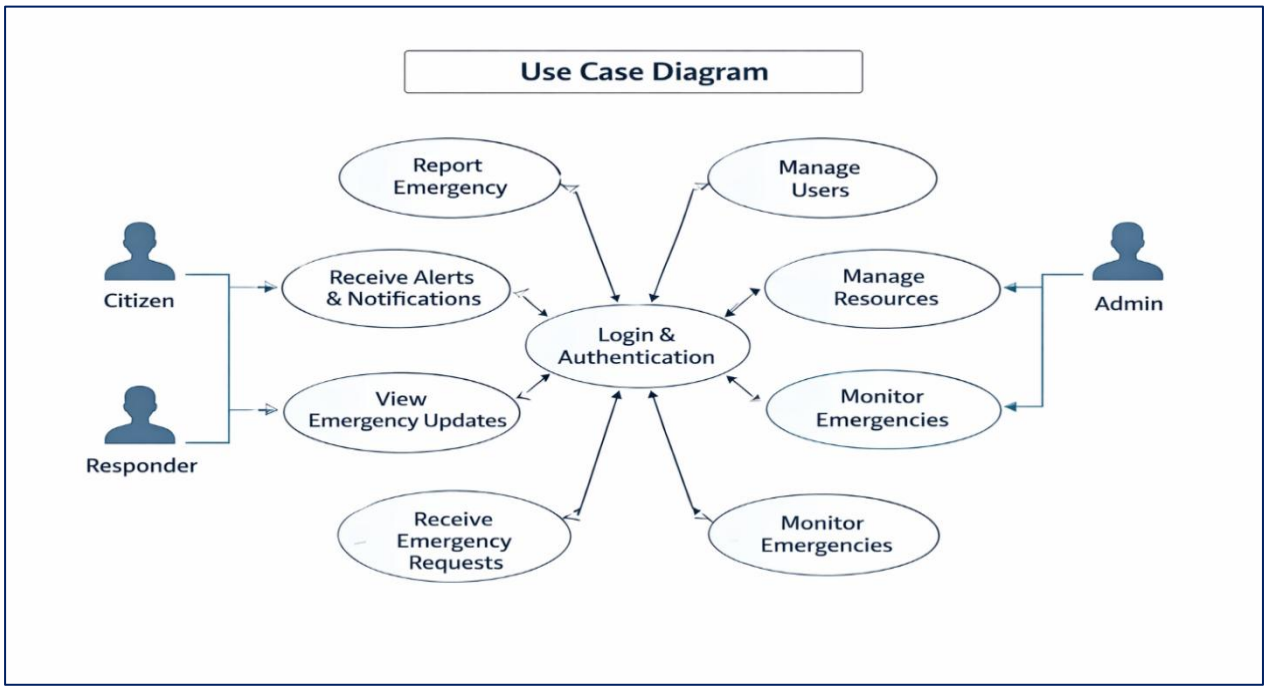

Figure 3.4.1 : Use Case Diagram

CHAPTER 4

METHODOLOGY

4.1 Working Principle

1. User registers and logs in
2. JWT token is generated
3. User reports emergency
4. Emergency is stored with “Pending” status
5. Admin updates status
6. User tracks progress

4.2 Process Flow

Authentication:

1. Input credentials
2. Validate user
3. Generate JWT
4. Return token

Emergency Creation:

1. Verify token
2. Accept input
3. Store emergency
4. Assign status

Status Update:

1. Verify admin
2. Update status
3. Save changes

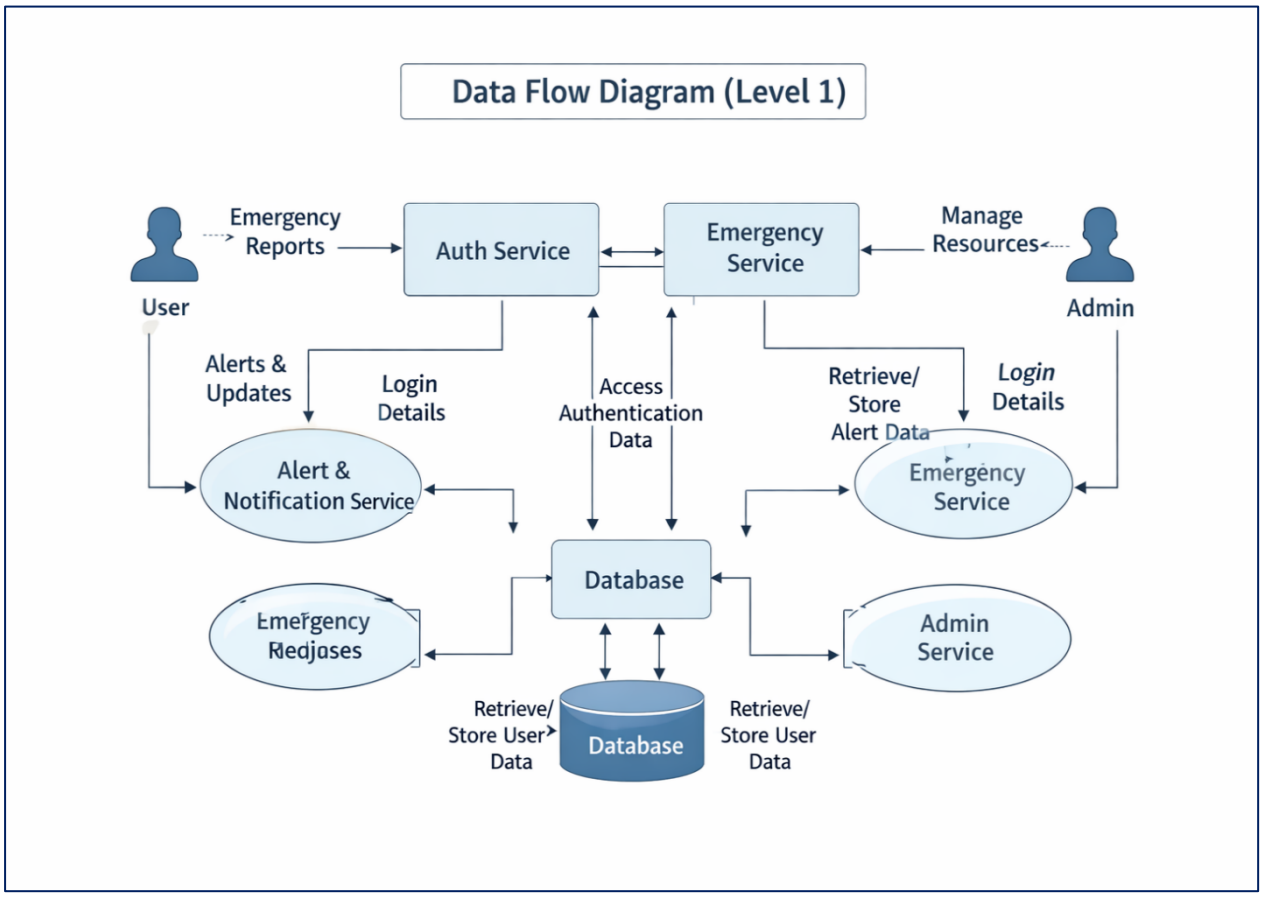

Figure 4.2.1 : Data Flow Diagram (Level 1)

4.3 Module Description

Auth Module:

- Registration
- Login
- Token generation

Emergency Module:

- Create emergency
- View emergencies
- Update status

Admin Module:

- View users
- Monitor emergencies
- Update records

API Gateway:

- Routes all requests
- Handles communication

4.4 Error Handling and Validation

- Input validation for forms
- Duplicate user prevention
- Token verification errors
- Unauthorized access handling
- Server error handling

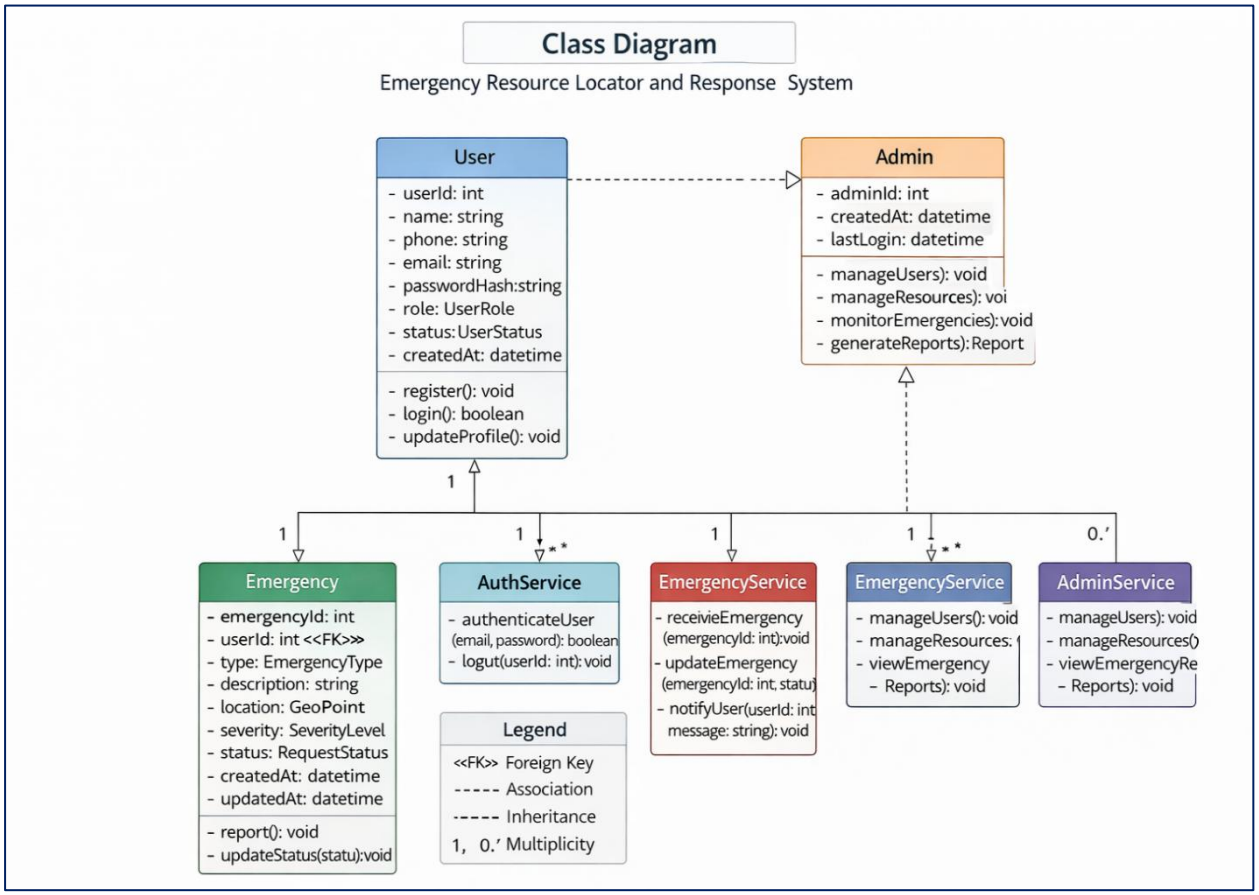

Figure 4.4.1 : of Emergency Resource Locator and Response System Class Diagram

CHAPTER 5

EXECUTION

5.1 Execution Steps

Step 1: Start MongoDB

Step 2: Run backend services

Step 3: Start API Gateway

Step 4: Run frontend

Step 5: Access application

5.2 Compilation & Execution Commands

Backend:

- npm install
- npm run dev

Frontend:

- npm install
- npm run dev

Docker:

- docker-compose up --build

CHAPTER 6

RESULTS

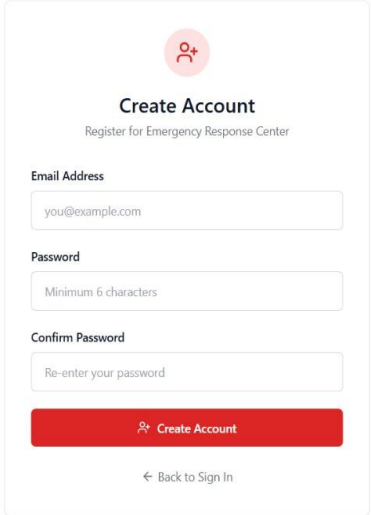

Figure 6.1.1 : User login interface.

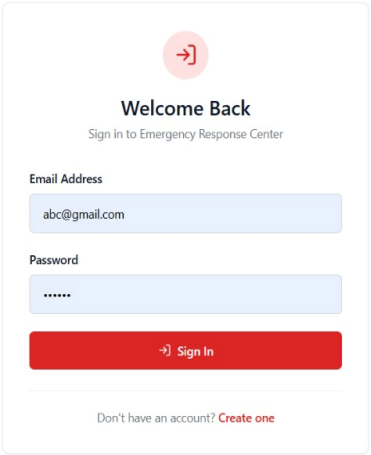

Figure 6.1.2 User registration interface

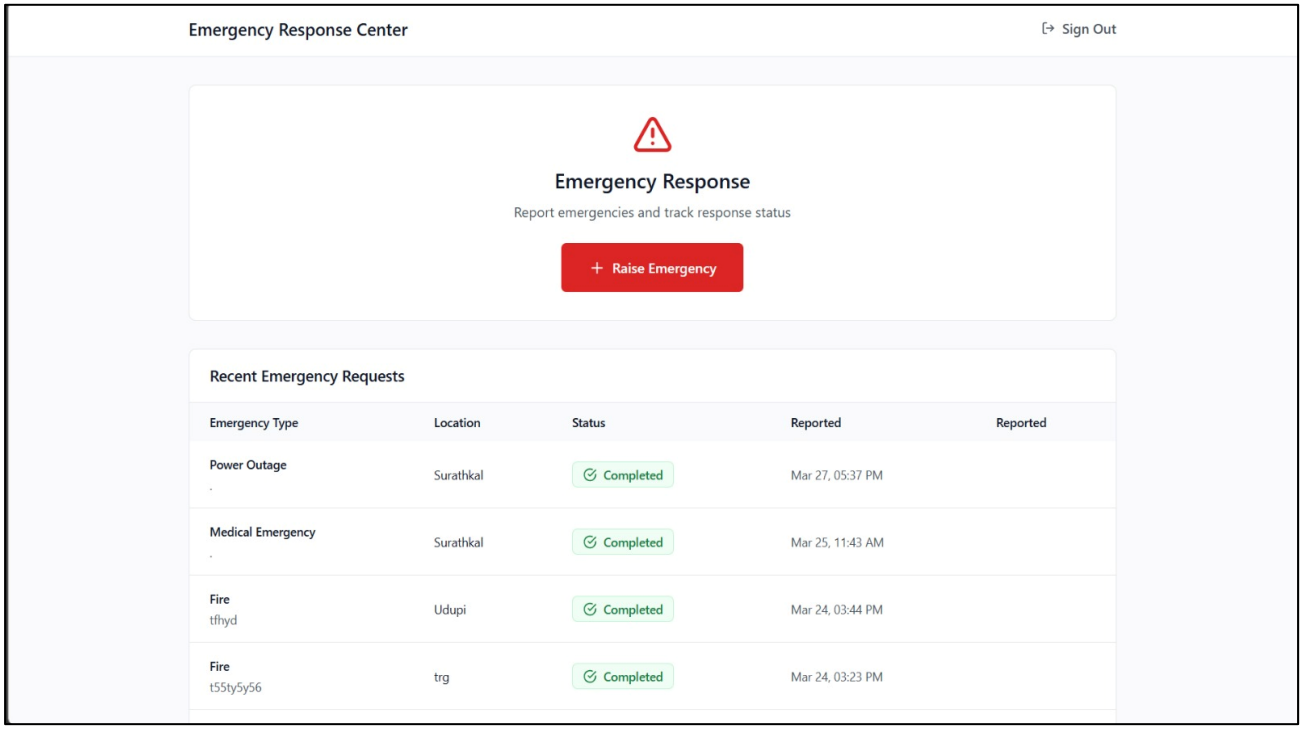

Figure 6.1.3 : Emergency requests dashboard.

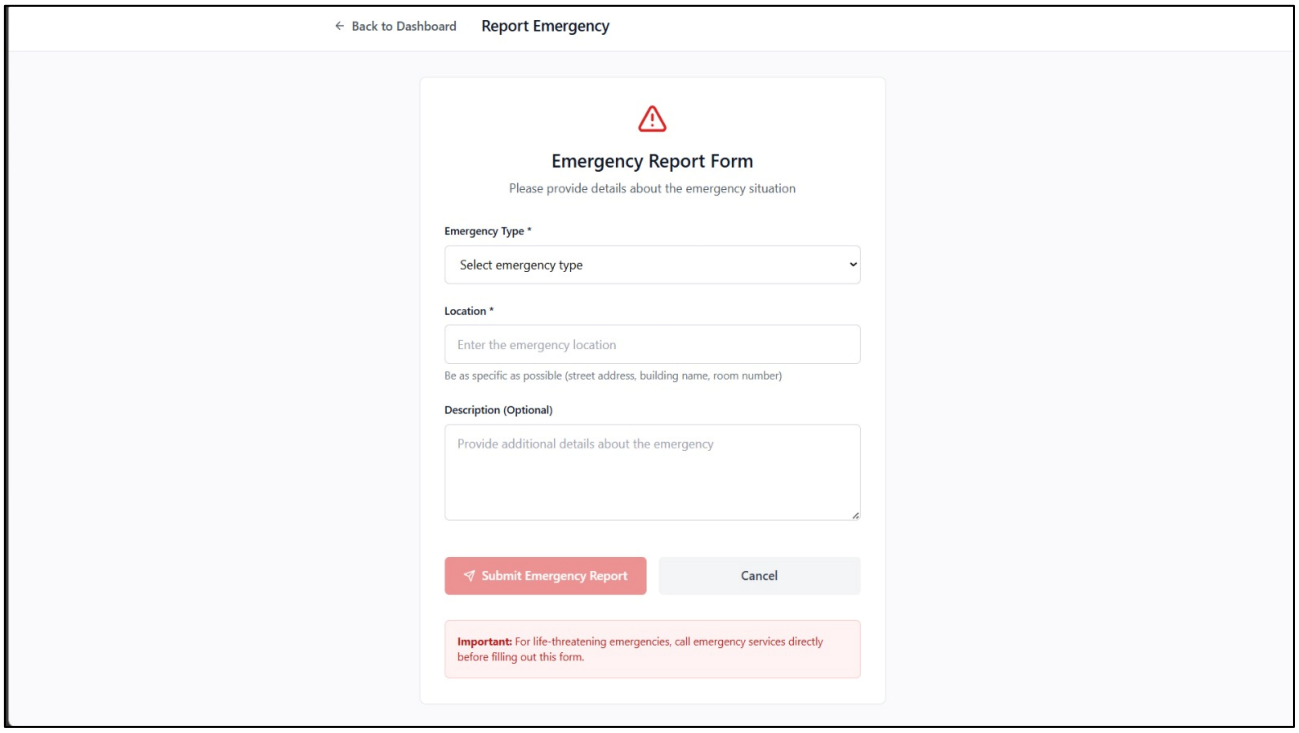

Figure 6.1.4 : Emergency report submission form.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Emergency Resource Locator and Response System successfully demonstrates the development of a scalable, secure, and efficient platform for emergency reporting and response coordination. The system utilizes a microservices architecture to ensure modularity, flexibility, and ease of maintenance.

By incorporating JWT-based authentication and role-based access control, the system ensures secure handling of user data and controlled access to functionalities. The use of an API Gateway enables seamless communication between services, improving system organization and performance.

The platform enhances communication between users and administrators, reduces response time during emergencies, and provides transparency through real-time status tracking. It also ensures efficient management of emergency requests through a structured and centralized approach.

Overall, the project achieves its objectives and serves as a practical implementation of modern distributed system design principles for solving real-world emergency management challenges.

7.2 Future Enhancements

- Real-time notifications (SMS/Email)
- GPS-based location tracking
- AI-based priority prediction
- Mobile application integration
- Advanced analytics dashboard

CHAPTER 8

REFERENCES

- Node.js, *Node.js Documentation*.
  Available at: https://nodejs.org/en/docs
- Express.js, *Express.js Official Documentation*.
  Available at: https://expressjs.com/
- MongoDB, *MongoDB Documentation*.
  Available at: https://www.mongodb.com/docs/
- Mongoose, *Mongoose Documentation*.
  Available at: https://mongoosejs.com/docs/
- React, *React Official Documentation*.
  Available at: https://react.dev/
- TypeScript, *TypeScript Documentation*.
  Available at: https://www.typescriptlang.org/docs/
- Tailwind CSS, *Tailwind CSS Documentation*.
  Available at: https://tailwindcss.com/docs
- Vite, *Vite Documentation*.
  Available at: https://vitejs.dev/guide/
- Docker, *Docker Documentation*.
  Available at: https://docs.docker.com/